

MEMOIR Depth Feed

V2.0a

This specification is being provided to you strictly for informational purposes solely for the purpose of developing or operating systems that interact with the 24 Exchange, or 24X. All proprietary rights and interest in this specification and the information contained herein shall be vested in 24X or its licensors and all other rights including, but without limitation, patent, registered design, copyright, trademark, service mark, connected with this publication shall also be vested in 24X or its licensors. No part of this specification may be redistributed or reproduced in any form or by any means or used to make any derivative work (such as translation, transformation, or adaptation) without written permission from 24X. 24X and its licensors reserve the right to withdraw, modify, or replace the specification at any time, without notice. No obligation is made by 24X regarding the level, scope, or timing of 24X's implementation of the functions or features discussed in this specification.

THE SPECIFICATION IS PROVIDED "AS IS", "WITH ALL FAULTS" AND 24X AND ITS LICENSORS MAKE NO WARRANTIES AND DISCLAIMS ALL WARRANTIES, EXPRESSED OR IMPLIED, OR STATUTORY RELATED TO THE SPECIFICATIONS. 24X AND ITS LICENSORS ARE NOT LIABLE FOR ANY INCOMPLETENESS OR INACCURACIES IN THE SPECIFICATIONS. 24X AND ITS LICENSORS ARE NOT LIABLE FOR ANY CONSEQUENTIAL, INCIDENTAL OR INDIRECT DAMAGES RELATING TO THE SPECIFICATIONS OR THEIR USE.

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

Table of Contents

1 Overview 6

2 Architecture 7

3 Encoding 8
3.1 Data Types 8
3.1.1 PriceType 8
3.1.2 String 8
3.1.3 BooleanType 9
3.1.4 UTC Timestamp Nanos 9
3.2 Header 9

4 Message Field Types 10
4.1 AuctionType 10
4.2 HaltReasonType 10
4.3 OrderFlagsType 11
4.4 SCExtendedHoursOrSoldType 11
4.5 SCSettlementType 11
4.6 SCSROTradeDetailType 11
4.7 SCTradeThroughExemptionType 12
4.8 SecurityTradingStatusType 12
4.9 SecurityTradingStatusReasonType 12
4.10 SideType 13
4.11 TradingSessionType 13

5 Messages 14
5.1 Instrument Directory 14
5.2 Reg SHO Restriction 14
5.3 Security Trading Status 15
5.4 Trading Session Status 16
5.5 Order Added 16
5.6 Order Deleted 16
5.7 Order Reduced 17
5.8 Order Executed 17
5.9 Trade 18
5.10 Broken Trade 18
5.11 Corrected Trade 19
5.12 Clear Book 19
5.13 Snapshot Complete 19
5.14 Auction Status 20
5.15 Auction Trade 21
5.16 Broken Auction Trade 22
5.17 Corrected Auction Trade 22

6 Message/State Recovery Methods 25
6.1 Gap Fill 25
6.2 Snapshot 26

7 Message Examples 28
7.1 Instrument Directory Message 28
7.2 Reg Sho Restriction Message 28
7.3 Security Trading Status Message 28
7.4 Order Added Message 29
7.5 Order Deleted Message 29
7.6 Order Reduced Message 29

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

7.7 Order Executed Message 29
7.8 Trade Message 30
7.9 Broken Trade Message 30
7.10 Corrected Trade Message 30
7.11 Clear Book Message 31
7.12 Snapshot Complete Message 31
7.13 Auction Status Message 31
7.14 Auction Trade Message 32
7.15 Broken Auction Trade Message 32
7.16 Corrected Auction Trade Message 33

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

< THIS PAGE INTENTIONALLY LEFT BLANK>

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

1 Overview

MEMOIR Depth Feed is a real-time depth of book feed.

The MEMOIR Depth Feed provides an event based binary messaging protocol that supports the following functionality:

- **Trading Session Information** - Provides the state of the trading session.
- **Order Data** - Provides the complete order book containing all displayed orders.
- **Non Displayed Trades** - Provides trade information for executions on non-displayed orders.
- **Instrument Directory** - Provides a mapping of all supported security IDs to all their basic attributes.
- **Trading Actions** - Supplies trading status messages to inform participants of events for an instrument.

The MEMOIR Depth Feed is unidirectional and cannot be used to enter orders.

Application messages are implemented using a binary protocol based on SBE (Simple Binary Encoding).

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

2 Architecture

The MEMOIR Depth Feed is a sequenced message stream using fixed width binary messages.

The business level messaging is transported via the following protocols for framing and recovery:

- MEMX-UDP protocol - UDP Multicast based session transport for the real-time delivery of messages.
- MEMX-TCP protocol - TCP based gap fill replay service.

At the start of the day the a session ID will be provided, and messages begin at sequence number 1. An Instrument Directory message spin will be sent out to map all the traded symbols to a symbol locate code that will be used throughout the session. The locate code will not change during the course of the session, however intraday there may be subsequent updates to the same or new instruments.

In order to build the book properly all orders will be sent keyed upon an OrderID. Events on the order (Add, Replace, Reduce, Executed) will alter the order on the book. It is the responsibility of the client to organize their representation of the order book based upon time, side, size and price.

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

3 Encoding

The MEMOIR Depth Feed uses the FIX Trading Community's Simple Binary Encoding (SBE) to specify message encoding. More information about SBE can be found at the FIX SBE XML Primer.

The MEMOIR Depth Feed is always encoded in Big Endian byte order.

3.1 Data Types

All encoding and decoding for SBE is centered around a set of basic primitive types.

For more information on primitive type encoding, see the SBE specification.

Type	Length (bytes)	Description	Value Range	Null Value	Null Value (Hex)
CHAR	1	ASCII Character	0 (NUL) to 127 (DEL)	0	0x00
INT8	1	Signed Integer	-127 to 127	-128	0x80
INT16	2	Signed Integer	-32767 to 32767	-32768	0x8000
INT32	4	Signed Integer	-2^31 + 1 to 2^31 - 1	-2^31 (-2147483648)	0x80000000
INT64	8	Signed Integer	-2^63 + 1 to 2^63 - 1	-2^63	0x8000000000000000
UINT8	1	Unsigned Integer	0 to 254	255	0xFF
UINT16	2	Unsigned Integer	0 to 65534	65535	0xFFFF
UINT32	4	Unsigned Integer	0 to 2^32-2	2^32 - 1 (4294967295)	0xFFFFFFFF
UINT64	8	Unsigned Integer	0 to 2^64 - 2	2^64 - 1	0xFFFFFFFFFFFFFFFF

The MEMOIR specification does not use the SBE floating point data types.

3.1.1 PriceType

Prices are encoded as a fixed-point scaled decimal, consisting of a signed long (int64) mantissa and a constant exponent of -6.

Type Name	Length	Type	Description
Mantissa	8	INT64	The fixed-point decimal representation of the price.
Exponent	N/A	INT8	MEMOIR uses a constant exponent of -6 for all prices.

For example, a mantissa of 1234567 with the constant exponent of -6 represents the decimal number 1.234567, and would appear encoded on the wire as the hex value 000000000012D687.

3.1.2 String

Strings are fixed length ASCII based character sequences. Lengths are defined in the schema. Variable length fields are not supported.

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

3.1.3 BooleanType

Boolean values are not defined specifically in SBE. The schema defines a BooleanType which represents a numeric value with True = `1` and False = `0`.

3.1.4 UTC Timestamp Nanos

Fields with the `UTCTimestampNanos` type represents a timestamp in Coordinated Universal Time (UTC) which begins at the UNIX epoch (January 1, 1970 at 00:00:00 UTC).

UTCTimestampNanos has the time unit of nanoseconds and is encoded as follows:

Type Name	Length	Type	Description
Time	8	UINT64	UTC timestamp since unix epoch with nanosecond precision.
Unit	N/A	UINT8	Unit of time. UTCTimestampNanos are represented in nanoseconds. This field is constant (value=`9`) and as such will not be transferred on the wire.

3.2 Header

SBE includes a header for each message. The SBE header is followed by the SBE body for the message.

The common SBE message header contains the fields that allows the decoder to identify what codec should be used as the template for a message.

1. **blockLength**: The number of bytes in the message body (does not include the header bytes).
2. **templateID**: The identifier for the template type of the message that is to follow.
3. **schemaID**: The identifier for the schema the message belongs to.
4. **version**: The version of the schema allowing for extension. Two pieces of information are packed into the (UINT16) version field, a major version and a minor update version. So for example a version of 258 = 0x0102 indicates major version 1, minor update 2.

The MEMOIR SBE header appears on the wire as:

Field	Offset	Length	Type	Description
BlockLength	0	2	UINT16	The number of bytes in the message body (does not include the header bytes). Note that MEMOIR messages do not use repeating groups or variable-length fields.
TemplateID	2	1	UINT8	Identifier of the message template (ie. the message type).
SchemaID	3	1	UINT8	The identifier of a message schema. SchemaId=`2` for MEMOIR Depth.
Version	4	2	UINT16	The version number of the message schema that was used to encode a message. Two pieces of information are packed into the (UINT16) version field, a major version and a minor update version. So for example a version of 258 = 0x0102 indicates major version 1, minor update 2.

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

4 Message Field Types

All messages are composed of fields. Each field has a type. This section defines the field types, their underlying on the wire type, acceptable values and a description of the type.

4.1 AuctionType

AuctionType represents the type of auction.

AuctionType is a 1-byte CHAR value.

Value	Name	Description
`Space ' ' (ASCII character 0x20)`	Unknown	Default auction type
`O`	Opening	Opening auction
`C`	Closing	Closing auction
`H`	Halt	Regulatory halt auction
`I`	IPO	IPO auction
`P`	Pause	Volatility pause auction

Messages that contains this field type include: Auction Status and Auction Trade

4.2 HaltReasonType

HaltReasonType represents the reason for halt.

HaltReasonType is a 1-byte UINT8 value.

Value	Name
0	Unknown
1	Admin
2	AdditionalInfoRequested
3	RegulatoryConcern
4	NewsReleased
5	MergerEffective
6	ETFComponentPricesNotAvailable
7	OrderImbalance
8	LULDTradingPause
9	CorporateAction
10	NewSecurityOffering
11	NewsPending
12	IntradayIndicativeValueNotAvailable
13	Operational
14	SubPennyTrading
15	MarketWideCircuitBreakerLevel1
16	MarketWideCircuitBreakerLevel2
17	MarketWideCircuitBreakerLevel3

Messages that contain this field type include: Security Trading Status

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

4.3 OrderFlagsType

OrderFlagsType represents additional indicators for an order.

OrderFlagsType is a 1-byte UINT8 value.

Type Name	Length	Type	Values	Description
OrderFlagsType	1	UINT8	Bit 0 - RetailPresentOnOrder	Enable an order flag by setting a bit. Disable an order flag by clearing the bit. Bit 0 is the least significant bit.

Messages that contain this field type include: Order Added

4.4 SCExtendedHoursOrSoldType

SCExtendedHoursOrSoldType represents the extended trading hour sale condition from the CTS.

SCExtendedHoursOrSoldType is a 1-byte CHAR value.

Value	Name
`T`	Form-T
`Space ' ' (ASCII character 0x20)`	Not applicable

Messages that contain this field type include: Auction Trade

4.5 SCSettlementType

SCSettlementType represents Level-1 sale condition as transmitted by the UTP (Unlisted Trading Privileges) and CTS (Consolidated Trading System).

SCSettlementType is a 1-byte CHAR value.

Value	Name
`@`	Regular Trade

Messages that contain this field type include: Auction Trade

4.6 SCSROTradeDetailType

SCSROTradeDetailType represents the self-regulatory organization trade reporting category.

By default, this value will be `Not Applicable`.

SCSROTradeDetailType is a 1-byte CHAR value.

Value	Name
`H`	Price Variation Trade
`I`	Odd Lot Trade
`X`	Auction Trade
`Space ' ' (ASCII character 0x20)`	Not Applicable

Messages that contain this field type include: Auction Trade

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

4.7 SCTradeThroughExemptionType

SCTradeThroughExemptionType represents the exemption rule which allowed a trade-through.

SCTradeThroughExemptionType is a 1-byte CHAR value.

Value	Name
O	Market Center Opening Trade (used in conjunction with X in the SCSROTradeDetailType field to denote the type of Auction Trade)
5	Market Center Reopening Trade (used in conjunction with X in the SCSROTradeDetailType field to denote the type of Auction Trade)
6	Market Center Closing Trade (used in conjunction with X in the SCSROTradeDetailType field to denote the type of Auction Trade)
Space ' ' (ASCII character 0x20)	Not Applicable

Messages that contain this field type include: Auction Trade

4.8 SecurityTradingStatusType

SecurityTradingStatusType represents the current security trading state of a symbol on the exchange.

SecurityTradingStatusType is a 1-byte CHAR value.

Value	Name
H	Halted
P	Paused
Q	Quoting
T	Trading

Messages that contain this field type include: Security Trading Status

4.9 SecurityTradingStatusReasonType

SecurityTradingStatusReasonType represents the reason for this security status.

SecurityTradingStatusReasonType is a 1-byte CHAR value.

Value	Name	Description
X	None	The security trading status does not apply (e.g the security is trading normally).
R	Regulatory	The security trading status originated from a regulator or the primary listing exchange for the security and was received via the SIP.
A	Administrative	The security trading status change originated from the exchange.

Messages that contain this field type include: Security Trading Status

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

4.10 SideType

SideType describes the side the order is on: either bid (buy) or offer (sell).

SideType is a 1-byte CHAR value.

Value	Name
B	Buy
S	Sell/SellShort/SellShortExempt

Messages that contain this field type include: Order Added

4.11 TradingSessionType

TradingSessionType represents the current trading session of the exchange.

TradingSessionType is a 1-byte CHAR value.

Value	Name	Description
0	Overnight	Overnight session
1	Opening	Pre-Market session
2	Trading	Market session
3	PostTrading	Post-Market session
4	Closed	Market closed

Messages that contain this field type include: Trading Session Status.

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

5 Messages

This section defines the messages that make up the protocol. For each message, it lists the fields in the message, as well as each field's position and length in the message, its underlying type, and a description of its purpose.

5.1 Instrument Directory

At the start of the trading day, an instrument directory message will be sent for all tradable securities on the exchange. In order to reduce the message sizes and improve performance, a SecurityID will be used in place of a ticker symbol on subsequent messages. SecurityIDs will start at 1 and increment. Do not rely on SecurityIDs being the same in subsequent trading days. The exchange uses CMS as its trading symbology, which requires a root and suffix combination.

Field	Offset	Length	Type	Meaning
SBE Header	0	6	N/A	SBE Header with SchemaId=2, TemplateId=1, BlockLength=27
Timestamp	6	8	UTCTimestampNanos	The timestamp when the event occurred.
SecurityID	14	2	UINT16	A unique code identifying the instrument for any messages in this session.
Symbol	16	6	String	The CMS root for the tradable instrument.
SymbolSfx	22	6	String	The CMS suffix for the tradable instrument.
RoundLot	28	4	UINT32	The number of shares in a round lot for the instrument.
IsTestSymbol	32	1	BooleanType	Determines if this security is a test instrument.

5.2 Reg SHO Restriction

In 2010, the Securities and Exchange Commission adopted Rule 201 of Regulation SHO. Rule 201 restricts the price at which short sales may occur when a stock has experienced significant downward price pressure. Rule 201 is designed to prevent short selling, including potentially manipulative or abusive short selling, from driving down further the price of a security that has already experienced a significant intra-day price decline, and to facilitate the ability of long sellers to sell first upon such a decline.

This message is used to indicate when a specific security is subject to the short sale price restriction. If this message is sent and the ShortSaleRestriction flag is set to true, Rule 201 is now in effect for that security ID.

NOTE: This message may be sent immediately after the related `Instrument Directory` message is sent for the security if the restriction is already in effect at the start of the MEMOIR data session.

Field	Offset	Length	Type	Meaning
SBE Header	0	6	N/A	SBE Header with SchemaId=2, TemplateId=2, BlockLength=11

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

Field	Offset	Length	Type	Meaning
Timestamp	6	8	UTCTimestampNanos	The timestamp when the event occurred.
SecurityID	14	2	UINT16	An instrument code uniquely identifying the security from the Instrument Directory.
ShortSaleRestriction	16	1	BooleanType	Identifies if this security is subject to the Regulation SHO short sales restrictions.

5.3 Security Trading Status

This message will inform the client of the current trading status of a security. The trading status of a security may change due to the following reasons:

- The primary market for a security has the authority to halt or pause a security via the SIP based upon upcoming news dissemination, or for regulatory purposes such as LULD threshold violations.
- The exchange may internally halt trading for a security for administrative or operational reasons.

Security Trading Status messages are sent out after Instrument Directory messages have been sent.

This message can and will be sent out throughout the trading session to indicate realtime changes in the security state.

If a `SecurityTradingStatus` message is not received for a security, it should be assumed that the status is `Halted`.

Field	Offset	Length	Type	Meaning
SBE Header	0	6	N/A	SBE Header with SchemaId=2, TemplateId=3, BlockLength=13
Timestamp	6	8	UTCTimestampNanos	The timestamp when the event occurred.
SecurityID	14	2	UINT16	An instrument code uniquely identifying the security from the Instrument Directory.
SecurityTradingStatus	16	1	SecurityTradingStatusType	The current trading state.
SecurityTradingStatusReason	17	1	SecurityTradingStatusReasonType	The source of the trading status change.

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

Field	Offset	Length	Type	Meaning
HaltReason	18	1	HaltReasonType	Halt reason for listed symbols

5.4 Trading Session Status

The exchange has entered a new trading session.

Field	Offset	Length	Type	Meaning
SBE Header	0	6	N/A	SBE Header with SchemaId=2, TemplateId=5, BlockLength=9
Timestamp	6	8	UTCTimestampNanos	The timestamp when the event occurred.
TradingSession	14	1	TradingSessionType	The trading session which was entered.

5.5 Order Added

A visible order has been added to the book.

The associated OrderID on this message will be used as the key that may be used to track this order's lifecycle in subsequent messages.

Field	Offset	Length	Type	Meaning
SBE Header	0	6	N/A	SBE Header with SchemaId=2, TemplateId=10, BlockLength=31
Timestamp	6	8	UTCTimestampNanos	The timestamp when the event occurred.
SecurityID	14	2	UINT16	An instrument code uniquely identifying the security from the Instrument Directory.
OrderID	16	8	UINT64	The trading session unique reference number assigned to the order.
Side	24	1	SideType	The side of the order being added.
Quantity	25	4	UINT32	The total quantity being added.
Price	29	8	PriceType	The displayed price of the new order.
OrderFlags	37	1	OrderFlagsType	Additional indicators for the order.

5.6 Order Deleted

A visible order was removed from the book. This can be due to a cancellation requested by the customer, or due to an administrative operation.

This OrderID will no longer be tracked, and should be removed from the book.

Field	Offset	Length	Type	Meaning
SBE Header	0	6	N/A	SBE Header with SchemaId=2, TemplateId=11, BlockLength=18
Timestamp	6	8	UTCTimestampNanos	The timestamp when the event occurred.

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

Field	Offset	Length	Type	Meaning
SecurityID	14	2	UINT16	An instrument code uniquely identifying the security from the Instrument Directory.
OrderID	16	8	UINT64	The trading session unique reference number assigned to the order in the Order Added message.

5.7 Order Reduced

A visible order was reduced in quantity while maintaining its matching priority in the book. This can be due to a replace or partial cancel request received from the customer.

The quantity field must be subtracted from the existing order quantity, with the remainder remaining on the book. If the value resulting from the subtraction reduce the order quantity to zero the order should be removed from the book.

Field	Offset	Length	Type	Meaning
SBE Header	0	6	N/A	SBE Header with SchemaId=2, TemplateId=12, BlockLength=22
Timestamp	6	8	UTCTimestampNanos	The timestamp when the event occurred.
SecurityID	14	2	UINT16	An instrument code uniquely identifying the security from the Instrument Directory.
OrderID	16	8	UINT64	The trading session unique reference number assigned to the order in the Order Added message.
Quantity	24	4	UINT32	The total quantity being reduced. This value will always be greater than 0.

5.8 Order Executed

A visible order on the book was executed.

If the quantity executed reduces the remaining quantity of the order to 0, the order should be removed from the book. Otherwise, the remaining order quantity after the execution remains on the order book with the same time-priority.

The executed price given by this message may differ from the original displayed price of the order, due to system price improvement or price sliding.

Field	Offset	Length	Type	Meaning
SBE Header	0	6	N/A	SBE Header with SchemaId=2,TemplateId=13, BlockLength=38
Timestamp	6	8	UTCTimestampNanos	The timestamp when the event occurred.
SecurityID	14	2	UINT16	An instrument code uniquely identifying the security from the Instrument Directory.

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

Field	Offset	Length	Type	Meaning
OrderID	16	8	UINT64	The trading session unique reference number assigned to the order in the Order Added message.
TradeID	24	8	UINT64	A trading session unique match number of the trade.
Quantity	32	4	UINT32	The total quantity executed.
Price	36	8	PriceType	The executed price of the order, it may not be the price that it was entered at due various trading methodologies.

5.9 Trade

A non-displayed order on the book was executed.

This message does not modify the state of the displayed order book, but does impact cumulative volume and average price calculations.

Field	Offset	Length	Type	Meaning
SBE Header	0	6	N/A	SBE Header with SchemaId=2, TemplateId=14, BlockLength=30
Timestamp	6	8	UTCTimestampNanos	The timestamp when the event occurred.
SecurityID	14	2	UINT16	An instrument code uniquely identifying the security from the Instrument Directory.
TradeID	16	8	UINT64	A trading session unique match number of the trade.
Quantity	24	4	UINT32	The total quantity traded.
Price	28	8	PriceType	The price of the matched execution.

5.10 Broken Trade

An operational or regulatory action has resulted in an execution being busted. Trades cannot be reinstated once they have been reported broken.

Broken trades should be removed from daily volume and average price calculations.

Field	Offset	Length	Type	Meaning
SBE Header	0	6	N/A	SBE Header with SchemaId=2, TemplateId=15, BlockLength=30
Timestamp	6	8	UTCTimestampNanos	The timestamp when the event occurred.
SecurityID	14	2	UINT16	An instrument code uniquely identifying the security from the Instrument Directory.
TradeID	16	8	UINT64	The trading session unique match number assigned to the original trade by the Trade or Order Executed message.
OriginalQuantity	24	4	UINT32	The original quantity of the trade being broken.
OriginalPrice	28	8	PriceType	The original price of the trade being broken.

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

5.11 Corrected Trade

An operational or regulatory action has resulted in an execution being corrected, without busting the entire trade.

Corrected trades should result in the original volume and price being removed from daily volume and average price calculations, and replaced by the new values given in this message.

Field	Offset	Length	Type	Meaning
SBE Header	0	6	N/A	SBE Header with SchemaId=2, TemplateId=16, BlockLength=42
Timestamp	6	8	UTCTimestampNanos	The timestamp when the event occurred.
SecurityID	14	2	UINT16	An instrument code uniquely identifying the security from the Instrument Directory.
TradeID	16	8	UINT64	The trading session unique match number assigned to the original trade by the Trade or Order Executed message.
OriginalQuantity	24	4	UINT32	The original quantity of the trade being corrected.
OriginalPrice	28	8	PriceType	The original price of the trade being corrected.
CorrectedQuantity	36	4	UINT32	The corrected quantity of the trade being corrected.
CorrectedPrice	40	8	PriceType	The new price of the trade being corrected.

5.12 Clear Book

An operational or regulatory action has resulted in the book state being reset.

Consumers processing this message should remove all existing orders from the book for the security ID specified.

Field	Offset	Length	Type	Meaning
SBE Header	0	6	N/A	SBE Header with SchemaId=2, TemplateId=18, BlockLength=10
Timestamp	6	8	UTCTimestampNanos	The timestamp when the event occurred.
SecurityID	14	2	UINT16	An instrument code uniquely identifying the security from the Instrument Directory.

5.13 Snapshot Complete

End of the snapshot message, all messages have been sent. Used only in Snapshot recovery (see Message/State Recovery Methods below)

Upon receipt of this message, clients should disconnect from the Depth of Book Snapshot feed and follow the recovery procedure to reconcile their state against the incremental Depth of Book feed.

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

Field	Offset	Length	Type	Meaning
SBE Header	0	6	N/A	SBE Header with SchemaId=2, TemplateId=100, BlockLength=16
Timestamp	6	8	UTCTimestampNanos	The timestamp when the event occurred.
AsOfSequenceNumber	14	8	UINT64	The sequence number on the real-time multicast channel that this snapshot is based upon. All messages buffered from the real-time feed that are less than or equal to this number should be discarded. All messages buffered after this number should be applied. (with appropriate gaps filled normally).

5.14 Auction Status

This message contains all the relevant info about auction for a specific symbol during the quoting period.

Field	Offset	Length	Type	Meaning
SBE Header	0	6	N/A	SBE Header with SchemaId=2, TemplateId=19, BlockLength=87
Timestamp	6	8	UTCTimestampNanos	The timestamp when the event occurred.
SecurityID	14	2	UINT16	An instrument code uniquely identifying the security from the Instrument Directory.
AuctionID	16	8	UINT64	The trading session unique reference number assigned to the auction.
AuctionType	24	1	AuctionType	Auction type.
TieBreakerPrice	25	8	PriceType	Price used to break a tie if the auction price calculation evaluates to two prices with equal shares executed and equal imbalance quantities.
LowerCollarPrice	33	8	PriceType	Lower collar price that determines if an order is Auction Eligible or not.
UpperCollarPrice	41	8	PriceType	Upper collar price that determines if an order is Auction Eligible or not.
ReferencePrice	49	8	PriceType	The price within the Collar Price Range at which only

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

Field	Offset	Length	Type	Meaning
				Auction Eligible Auction-Only orders would match.
ReferenceBuyQuantity	57	8	UINT64	Total number of buy shares of **Auction Eligible** orders priced at or greater than the reference price.
ReferenceSellQuantity	65	8	UINT64	Total number of sell shares of **Auction Eligible** orders priced at or less than the reference price.
IndicativePrice	73	8	PriceType	The price at which Auction-Only and Continuous book orders would match if the auction were to occur right now.
AuctionOnlyPrice	81	8	PriceType	The price at which all Auction-Only orders would match.
NumberOfExtensions	89	4	UINT32	The number of times that the auction has been extended.

5.15 Auction Trade

The bulk auction trade for a symbol, contains total number of shares done and the price the auction occurred at.

Field	Offset	Length	Type	Meaning
SBE Header	0	6	N/A	SBE Header with SchemaId=2, TemplateId=20, BlockLength=47
Timestamp	6	8	UTCTimestampNanos	The timestamp when the event occurred.
SecurityID	14	2	UINT16	An instrument code uniquely identifying the security from the Instrument Directory.
AuctionID	16	8	UINT64	The trading session unique reference number assigned to the auction.
AuctionType	24	1	AuctionType	Auction type.
AuctionQuantity	25	8	UINT64	Total number of shares matched.
AuctionPrice	33	8	PriceType	The price of the matched execution.
TradeID	41	8	UINT64	A trading session unique match number of the trade.
SaleCondition1	49	1	SCSettlementType	The sale condition code level 1 - Settlement Type.

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

Field	Offset	Length	Type	Meaning
SaleCondition2	50	1	SCTradeThroughExemptionType	The sale condition code level 2 - Trade Through Exemption Type.
SaleCondition3	51	1	SCExtendedHoursOrSoldType	The sale condition code level 3 - Extended Hours Type.
SaleCondition4	52	1	SCSROTradeDetailType	The sale condition code level 4 - SRO Trade Detail Type.

5.16 Broken Auction Trade

An operational or regulatory action has resulted in an auction trade being busted.

Field	Offset	Length	Type	Meaning
SBE Header	0	6	N/A	SBE Header with SchemaId=2, TemplateId=21, BlockLength=42
Timestamp	6	8	UTCTimestampNanos	The timestamp when the event occurred.
SecurityID	14	2	UINT16	An instrument code uniquely identifying the security from the Instrument Directory.
AuctionID	16	8	UINT64	The trading session unique reference number assigned to the original auction trade being broken.
TradeID	24	8	UINT64	The trading session unique match number assigned to the original auction trade being broken
OriginalAuctionQuantity	32	8	UINT64	The original auction quantity of the auction trade being broken.
OriginalAuctionPrice	40	8	PriceType	The original auction price of the auction trade being broken.

5.17 Corrected Auction Trade

An operational or regulatory action has resulted in an auction trade being corrected.

Field	Offset	Length	Type	Meaning
SBE Header	0	6	N/A	SBE Header with SchemaId=2, TemplateId=22,

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

Field	Offset	Length	Type	Meaning
				BlockLength=66
Timestamp	6	8	UTCTimestampNanos	The timestamp when the event occurred.
SecurityID	14	2	UINT16	An instrument code uniquely identifying the security from the Instrument Directory.
AuctionID	16	8	UINT64	The trading session unique reference number assigned to the original auction trade being corrected.
TradeID	24	8	UINT64	The trading session unique match number assigned to the original auction trade being corrected.
OriginalAuctionQuantity	32	8	UINT64	The original auction quantity of the auction trade being corrected.
OriginalAuctionPrice	40	8	PriceType	The original auction price of the auction trade being corrected.
OriginalSaleCondition1	48	1	SCSettlementType	The original sale condition code level 1 - Settlement Type.
OriginalSaleCondition2	49	1	SCTradeThroughExemptionType	The original sale condition code level 2 - Trade Through Exemption Type.
OriginalSaleCondition3	50	1	SCExtendedHoursOrSoldType	The original sale condition code level 3 - Extended Hours Type.

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

Field	Offset	Length	Type	Meaning
OriginalSaleCondition4	51	1	SCSROTradeDetailType	The original sale condition code level 4 - SRO Trade Detail Type.
CorrectedAuctionQuantity	52	8	UINT64	The corrected auction quantity of the auction trade.
CorrectedAuctionPrice	60	8	PriceType	The corrected auction price of the auction trade.
CorrectedSaleCondition1	68	1	SCSettlementType	The corrected sale condition code level 1 - Settlement Type.
CorrectedSaleCondition2	69	1	SCTradeThroughExemptionType	The corrected sale condition code level 2 - Trade Through Exemption Type.
CorrectedSaleCondition3	70	1	SCExtendedHoursOrSoldType	The corrected sale condition code level 3 - Extended Hours Type.
CorrectedSaleCondition4	71	1	SCSROTradeDetailType	The corrected sale condition code level 4 - SRO Trade Detail Type.

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

6 Message/State Recovery Methods

As MEMOIR Depth of Book is disseminated primarily over the MEMX-UDP transport protocol, consumers of the feed may periodically miss messages due to the unreliable nature of the UDP protocol. Two independent Multicast groups will be provided for standard A/B arbitration and quick recovery of messages dropped due to packet loss on a single channel. However, there are cases, such as a catastrophic system issue on the client's end, or the loss of the same message from both channels, where the client may need to request data to be re-sent. For these situations,two modes of recovery are available for users. The first is a "gap fill" mechanism, used to recover small numbers of messages, typically for small episodes of packet loss that affect the same messages on both Multicast channels. The second is a "snapshot" mechanism, where the current business state of the feed is sent, followed by a "live" sequence number for the client to begin processing the real-time MEMX-UDP channels.

6.1 Gap Fill

A client detects missing messages over the MEMX-UDP transport for MEMOIR Depth of Book by using the sequence number in the MEMX-UDP datagram header to determine if a gap in sequence numbers has occurred. In order to recover these messages, the client may use a connection to a Gap Fill Server via the MEMX-TCP Replay mode to request the missing messages.

NOTE: The client may initiate and maintain a connection to the MEMX-TCP Gap Fill Server early (to verify connectivity) and stay connected (while periodically sending heartbeats) until the functionality is needed.

The process by which a client recovers missing messages via the Gap Fill Server is as follows:

1. Issue a MEMX-TCP `ReplayRequest` to the MEMOIR Depth of Book Gap Fill Server, containing the `SessionID` received over the MEMX-UDP channel(s), the `NextSequenceNumber` of the first missing message, and the `Count` of messages required (including the sequence provided)
2. While the above request is being processed, the client should buffer any received MEMX-UDP multicast messages and not apply them until the missing sequence numbers have been recovered.
3. After the receipt of the MEMX-TCP Replay Request, the Gap Fill Server will send the following:
    a. A MEMX-TCP `ReplayBegin` message, with `NextSequenceNumber` set to the requested Sequence number, and a `PendingMessageCount` set to the number of messages to follow in the replay. (This may be less than the requested count - see NOTE below)
    b. The requested number of **MEMOIR Depth Of Book** messages (or less given the constraints listed above on request size), starting at the requested sequence number
    c. A MEMX-TCP `ReplayComplete` message
4. After the `ReplayComplete` message is received, the client is free to send another request (if this request did not satisfy the entire gap)
5. Once the client has received all of the missing messages from one or more replay requests to the MEMX-TCP Replay server, the client may then process the messages buffered from the MEMX-UDP channel, discarding duplicate sequence numbers and applying future sequence numbers in order.

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

NOTE: the replay provided may consist of fewer messages than requested due to some constraints on the size of the response. The number of messages in the replay response will be the minimum of:

- the requested count
- the configured maximum messages per request for the service
- the number of messages remaining from the requested start sequence to the highest published sequence

6.2 Snapshot

If a catastrophic failure occurs on the client side that requires a full state reset on the client, and the client does not wish to manually replay the entire day via multiple MEMX-TCP Replay requests, the client may opt to use the MEMX-TCP snapshot mechanism to recover state. Once the snapshot has completed, the client should use normal Gap Fill processing (described above) to recover any further gaps in data that may occur during that trading session, as Snapshots can be quite large compared to a gap of one or two datagrams.

NOTE: The client may initiate and maintain a connection to the MEMX-TCP snapshot server early (to verify connectivity) and stay connected (while periodically sending heartbeats) until the functionality is needed.

The process by which a client recovers via snapshot is as follows:

1. Join the real-time MEMX-UDP multicast group(s) for the data feed, and begin buffering all received messages.
2. Issue a MEMX-TCP `ReplayAllRequest` to the MEMOIR Depth of Book snapshot server, containing the `SessionID` received over the MEMX-UDP channel(s)
3. After the receipt of the MEMX-TCP ReplayAll request, the Snapshot Server will send the following:
    a. A MEMX-TCP `ReplayBegin` message, with `NextSequenceNumber` set to 1 and `PendingMessageCount` set to the number of MEMOIR Depth of Book Business Level messages contained in the snapshot
    b. MEMOIR Depth of Book Business Level `InstrumentDirectory`messages containing a symbol description and associated SecurityID to be used throughout the session.
    c. MEMOIR Depth of Book Business Level `RegShoRestriction` messages containing the current Reg-SHO Restriction State for each symbol
    d. MEMOIR Depth of Book Business Level `SecurityTradingStatus` messages containing the current trading status for each symbol
    e. MEMOIR Depth of Book Business Level `TradingSessionStatus` message containing the current trading session status.
    f. MEMOIR Depth of Book Business Level `AddOrder` messages for each symbol
    g. MEMOIR Depth of Book Business Level `AuctionStatus` messages for each listed symbol only when an auction is in progress
    h. A single MEMOIR Depth of Book Business Level `SnapshotComplete` event containing an `AsOfSequenceNumber` set to the sequence number on the MEMX-UDP multicast channel that this snapshot was taken (clients should resume processing at this number + 1 on the multicast feed after applying the snapshot)

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

   i. A MEMX-TCP `ReplayComplete` message

4. Discard all buffered events from the MEMX-UDP real-time feed at or before the `AsOfSequenceNumber` in the received Snapshot Complete event.
5. Apply all buffered events from the MEMX-UDP real-time feed after the `AsOfSequenceNumber` in the received Snapshot Complete event, in sequence order.
6. Continue to process the MEMX-UDP real-time multicast feed normally from this point onward, using the **Gap Fill** mechanism above to recover dropped messages.

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

7 Message Examples

Note: the code fragments shown are representative of how to set field values in the SBE messages. Your interface may be be different.

7.1 Instrument Directory Message

```
instrumentDirectoryEncoder.timestamp().time(getTimeInNanos());
instrumentDirectoryEncoder.securityID(0xABCD);
instrumentDirectoryEncoder.symbol("AAPL");
instrumentDirectoryEncoder.roundLot(100);
instrumentDirectoryEncoder.isTestSymbol(BooleanType.False);

Hex code:
00000000: 001b0102 02000002 a6492d4e f02eabcd   |.........I-N....|
00000010: 4141504c 00000000 00000000 00000064   |AAPL...........d|
00000020: 00                                    |.|
```

7.2 Reg Sho Restriction Message

```
regShoRestrictionEncoder.timestamp().time(getTimeInNanos());
regShoRestrictionEncoder.securityID(0xABCD);
regShoRestrictionEncoder.shortSaleRestriction(BooleanType.True);

Hex code:
00000000: 000b0202 00010005 e17518f6 e552abcd   |.........u...R..|
00000010: 01                                    |.|
```

7.3 Security Trading Status Message

```
securityTradingStatusEncoder.timestamp().time(getTimeInNanos());
securityTradingStatusEncoder.securityID(0xABCD);
securityTradingStatusEncoder.securityTradingStatus(SecurityTradingStatusType.
Quoting);
securityTradingStatusEncoder.securityTradingStatusReason(SecurityTradingStatu
sReasonType.Regulatory);
securityTradingStatusEncoder.haltReason(HaltReasonType.Admin);

Hex code:
00000000: 000d0302 02000002 a686b821 a42aabcd   |...........!.*..|
00000010: 515201                                |QR.|
```

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

7.4 Order Added Message

```
orderAddedEncoder.timestamp().time(getTimeInNanos());
orderAddedEncoder.securityID(0xABCD);
orderAddedEncoder.orderId(0x1122334455667788L);
orderAddedEncoder.side(SideType.Buy);
orderAddedEncoder.quantity(1500);
orderAddedEncoder.price().mantissa(123450000);
orderAddedEncoder.orderFlags().retailPresent(true);

Hex code:
00000000: 00200a02 02000001 39706475 979eabcd   |. ......9pdu....|
00000010: 11223344 55667788 42000005 dc000000   |."3DUfw.B.......|
00000020: 00075bb2 9001                         |..[...|
```

7.5 Order Deleted Message

```
orderDeletedEncoder.timestamp().time(getTimeInNanos());
orderDeletedEncoder.securityID(0xABCD);
orderDeletedEncoder.orderId(0x1122334455667788L);

Hex code:
00000000: 00120b02 00010005 e175193b 96d1abcd   |.........u.;....|
00000010: 11223344 55667788                     |."3DUfw.|
```

7.6 Order Reduced Message

```
orderReducedEncoder.timestamp().time(getTimeInNanos());
orderReducedEncoder.securityID(0xABCD);
orderReducedEncoder.orderId(0x1122334455667788L);
orderReducedEncoder.quantity(2200);

Hex code:
00000000: 00160c02 00010005 e175194d 01f8abcd   |.........u.M....|
00000010: 11223344 55667788 00000898            |."3DUfw.....|
```

7.7 Order Executed Message

```
orderExecutedEncoder.timestamp().time(getTimeInNanos());
orderExecutedEncoder.securityID(0xABCD);
```

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

```
orderExecutedEncoder.orderId(0x1122334455667788L);
orderExecutedEncoder.tradeId(0xFFEEDDCCBBAA9988L);
orderExecutedEncoder.quantity(2100);
orderExecutedEncoder.price().mantissa(123450000);

Hex code:
00000000: 00260d02 00010005 e175195c acbaabcd   |.&.......u.\....|
00000010: 11223344 55667788 ffeeddcc bbaa9988   |."3DUfw.........|
00000020: 00000834 00000000 075bb290            |...4.....[..|
```

7.8 Trade Message

```
tradeEncoder.timestamp().time(getTimeInNanos());
tradeEncoder.securityID(0xABCD);
tradeEncoder.tradeId(0X112233);
tradeEncoder.quantity(200);
tradeEncoder.price().mantissa(123450000);

Hex code:
00000000: 001e0e02 00010005 e175196c 177eabcd   |.........u.l.~..|
00000010: 00000000 00112233 000000c8 00000000   |......"3........|
00000020: 075bb290                              |.[..|
```

7.9 Broken Trade Message

```
brokenTradeEncoder.timestamp().time(getTimeInNanos());
brokenTradeEncoder.securityID(0xABCD);
brokenTradeEncoder.tradeId(0x11223344L);
brokenTradeEncoder.originalQuantity(400);
brokenTradeEncoder.originalPrice().mantissa(123450000);

Hex code:
00000000: 001e0f02 00010005 e175197a 4cdaabcd   |.........u.zL...|
00000010: 00000000 11223344 00000190 00000000   |......"3D.......|
00000020: 075bb290                              |.[..|
```

7.10 Corrected Trade Message

```
correctedTradeEncoder.timestamp().time(getTimeInNanos());
correctedTradeEncoder.securityID(0xABCD);
correctedTradeEncoder.tradeID(0X112233);
correctedTradeEncoder.originalQuantity(200);
```

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

```
correctedTradeEncoder.originalPrice().mantissa(123450000);
correctedTradeEncoder.correctedQuantity(300);
correctedTradeEncoder.correctedPrice().mantissa(123470000);

Hex code:
00000000: 002a1002 00010005 e1751989 545cabcd   |.*.......u..T\..|
00000010: 00000000 00112233 000000c8 00000000   |......"3........|
00000020: 075bb290 0000012c 00000000 075c00b0   |.[.....,.....\..|
```

7.11 Clear Book Message

```
clearBookEncoder.timestamp().time(getTimeInNanos());
clearBookEncoder.securityID(0xABCD);

Hex code:
00000000: 000a1202 00010005 e1751996 dc46abcd   |.........u...F..|
```

7.12 Snapshot Complete Message

```
snapshotCompleteEncoder.timestamp().time(getTimeInNanos());
snapshotCompleteEncoder.asOfSequenceNumber(0x11223344L);

Hex code:
00000000: 00106402 00010005 e17519aa 085a0000   |..d......u...Z..|
00000010: 00001122 3344                         |..."3D|
```

7.13 Auction Status Message

```
auctionStatusEncoder.timestamp().time(getTimeInNanos());
auctionStatusEncoder.securityID(0xABCD);
auctionStatusEncoder.auctionID(0x11234);
auctionStatusEncoder.auctionType(AuctionType.Opening);
auctionStatusEncoder.tieBreakerPrice().mantissa(11234);
auctionStatusEncoder.lowerCollarPrice().mantissa(22334);
auctionStatusEncoder.upperCollarPrice().mantissa(33445);
auctionStatusEncoder.referencePrice().mantissa(445456);
auctionStatusEncoder.referenceBuyQuantity(100);
auctionStatusEncoder.referenceSellQuantity(200);
auctionStatusEncoder.indicativePrice().mantissa(66778);
auctionStatusEncoder.auctionOnlyPrice().mantissa(55667);
auctionStatusEncoder.numberOfExtensions(2);

Hex code:
```

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

```
00000000: 00571302 02000002 a686b86b 645fabcd   |.W.........kd_..|
00000010: 00000000 00011234 4f000000 0000002b   |.......4O......+|
00000020: e2000000 00000057 3e000000 00000082   |.......W>.......|
00000030: a5000000 000006cc 10000000 00000000   |................|
00000040: 64000000 00000000 c8000000 00000104   |d...............|
00000050: da000000 000000d9 73000000 02         |........s....|
```

7.14 Auction Trade Message

```
auctionTradeEncoder.timestamp().time(getTimeInNanos());
auctionTradeEncoder.securityID(0xABCD);
auctionTradeEncoder.auctionID(0x11234);
auctionTradeEncoder.auctionType(AuctionType.Opening);
auctionTradeEncoder.auctionQuantity(100);
auctionTradeEncoder.auctionPrice().mantissa(55667);
auctionTradeEncoder.tradeID(0x22334);
auctionTradeEncoder.saleCondition1(SCSettlementType.RegularTrade);
auctionTradeEncoder.saleCondition2(SCTradeThroughExemptionType.MarketCenterCl
osingTrade);
auctionTradeEncoder.saleCondition3(SCExtendedHoursOrSoldType.FormT);
auctionTradeEncoder.saleCondition4(SCSROTradeDetailType.AuctionTrade);

Hex code:
00000000: 002f1402 02000002 a686b873 bcc4abcd   |./.........s....|
00000010: 00000000 00011234 4f000000 00000000   |.......4O.......|
00000020: 64000000 000000d9 73000000 00000223   |d.......s......#|
00000030: 34403654 58                           |4@6TX|
```

7.15 Broken Auction Trade Message

```
brokenAuctionTradeEncoder.timestamp().time(getTimeInNanos());
brokenAuctionTradeEncoder.securityID(0xABCD);
brokenAuctionTradeEncoder.auctionID(0x11234);
brokenAuctionTradeEncoder.tradeID(0x22334);
brokenAuctionTradeEncoder.originalAuctionQuantity(100);
brokenAuctionTradeEncoder.originalAuctionPrice().mantissa(12345);

Hex code:
00000000: 002a1502 02000002 a686b886 0597abcd   |.*..............|
00000010: 00000000 00011234 00000000 00022334   |.......4......#4|
00000020: 00000000 00000064 00000000 00003039   |.......d......09|
```

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.

7.16 Corrected Auction Trade Message

```
correctedAuctionTradeEncoder.timestamp().time(getTimeInNanos());
correctedAuctionTradeEncoder.securityID(0xABCD);
correctedAuctionTradeEncoder.auctionID(0x11234);
correctedAuctionTradeEncoder.tradeID(0x22334);
correctedAuctionTradeEncoder.originalAuctionQuantity(100);
correctedAuctionTradeEncoder.originalAuctionPrice().mantissa(12345);
correctedAuctionTradeEncoder.originalSaleCondition1(SCSettlementType.RegularT
rade);
correctedAuctionTradeEncoder.originalSaleCondition2(SCTradeThroughExemptionTy
pe.MarketCenterClosingTrade);
correctedAuctionTradeEncoder.originalSaleCondition3(SCExtendedHoursOrSoldType
.FormT);
correctedAuctionTradeEncoder.originalSaleCondition4(SCSROTradeDetailType.Auct
ionTrade);
correctedAuctionTradeEncoder.correctedAuctionQuantity(200);
correctedAuctionTradeEncoder.correctedAuctionPrice().mantissa(12346);
correctedAuctionTradeEncoder.correctedSaleCondition1(SCSettlementType.Regular
Trade);
correctedAuctionTradeEncoder.correctedSaleCondition2(SCTradeThroughExemptionT
ype.MarketCenterClosingTrade);
correctedAuctionTradeEncoder.correctedSaleCondition3(SCExtendedHoursOrSoldTyp
e.FormT);
correctedAuctionTradeEncoder.correctedSaleCondition4(SCSROTradeDetailType.Auc
tionTrade);

Hex code:
00000000: 00421602 02000002 a686b88c b581abcd   |.B..............|
00000010: 00000000 00011234 00000000 00022334   |.......4......#4|
00000020: 00000000 00000064 00000000 00003039   |.......d......09|
00000030: 40365458 00000000 000000c8 00000000   |@6TX............|
00000040: 0000303a 40365458                     |..0:@6TX|
```

COPYRIGHT 24 EXCHANGE 2026. ALL RIGHTS RESERVED.